

MatForge

CIS 5650 Final Project

Cecilia Chen, Xiaonan Pan, Yiding Tian

The Problem & Our Approach

The Problem

Advanced material rendering requires a complete pipeline, but each stage has fundamental limitations:

- **Sampling Quality** → Standard Sobol' sequences have poor uniformity in consecutive 4D projections → slower convergence
- **Geometric Detail** → Displacement mapping requires millions of tessellated triangles → Memory explosion (500MB+ per 4K map)
- **Importance Sampling** → Standard VNDF sampling wastes 15-40% of samples on invalid directions → variance and wasted computation
- **Material Appearance** → Standard BRDFs (GGX) → Energy loss at high roughness → Unrealistic dark appearance
- **No existing solution** addresses all four stages together in real-time

Our Approach: Complete 4-Stage Pipeline

Combining FOUR complementary SIGGRAPH 2023-2024 papers:

1. **Quad-Optimized LDS** (SIGGRAPH 2024) - **Foundation**: Better Random Numbers
 - Optimized base-3 Sobol' sequences with **(1,4)-sequence** property
 - Faster convergence for path tracing (better sample uniformity)
2. **RMIP** (SIGGRAPH Asia 2023) - **Geometry**: Tessellation-Free Displacement
 - Tessellation-free displacement: **11× faster, 3× less memory** than traditional approaches
 - Direct ray tracing of displacement maps using custom intersection shaders
3. **Fast-MSX** (SIGGRAPH 2023) - **Materials**: Multiple Scattering
 - Physically-accurate multiple scattering: **100× better energy conservation, 5% overhead**
 - Closed-form solution for realistic rough material appearance
4. **Bounded VNDF** (SIGGRAPH Asia 2023) - **Sampling**: Efficient Importance Sampling
 - **15-40% variance reduction** for rough materials
 - Spherical cap bounding eliminates invalid samples

Combined Result: Foundation → Geometry → Sampling → Shading = **Complete production-quality material pipeline**

Why This Matters

Novel Contributions Beyond Paper Reproduction:

1. **First 4-Paper Combined Implementation:** No existing work combines Quad LDS + RMIP + Bounded VNDF + Fast-MSX
2. **First Vulkan Implementation:** All four papers used proprietary renderers or CPU-based reference code
3. **Production Workflow:** From random number generation to geometry to sampling and scattering, complete workflow improved

Industry-Relevant Problem:

- **AAA Games:** Memory bottleneck from tessellated displacement → RMIP solves this
- **Film/VFX:** Slow convergence from poor sampling → Quad LDS accelerates
- **Real-Time Rendering:** Inefficient BRDF sampling → Bounded VNDF reduces variance
- **Material Artists:** Unrealistic rough materials → Fast-MSX fixes energy loss

Real-World Impact:

- **Open-source material library** (7+ production-ready materials demonstrating all techniques)
- **Measurable improvements:** Convergence rate, memory reduction, variance reduction, energy conservation
- **Educational value:** Complete reference implementation for four recent SIGGRAPH papers
- **Potential publication:** First combined system, novel Vulkan optimizations

APIs & Platforms

Graphics APIs:

- Vulkan 1.3 (VK_KHR_ray_tracing_pipeline, VK_KHR_ray_tracing_maintenance1)
- Slang shader language (cross-compilation to SPIR-V)

Development Platform:

- C++, CMake build system
- Windows/Linux (cross-platform via Vulkan)

Target Hardware:

- NVIDIA RTX 4070 (team member GPU)
- Minimum: RTX 2060+ (any RTX GPU with ray tracing)

Third-Party Code & Attribution

Base Framework (nvpro-core2):

https://github.com/nvpro-samples/vk_gltf_renderer

- nvpro-samples/vk_gltf_renderer
- Vulkan glTF path tracer (Apache 2.0 License)
- nvpro-core2 framework - NVIDIA Professional Graphics utilities
- Will be clearly credited in README and code headers

Dependencies (auto-downloaded via CMake):

- Slang shader compiler
- RapidJSON for glTF parsing
- MikkTSpace for tangent calculation
- Optional: DRACO mesh compression, DLSS SDK

All third-party code is for infrastructure only:

Core contributions (RMIP algorithm, Fast-MSX BRDF, combined pipeline) are our original implementation.

Schedule

Milestone 1 (Nov 12): Individual Technique Foundations

- Cecilia: RMIP data structure builder + custom intersection shader working
- Yiding: Quad LDS generator matrices + device sampling integrated
- Xiaonan: Bounded VNDF sampler + basic Fast-MSX implementation

Milestone 2 (Nov 24): Integration & Material Library

- All 4 techniques working together: Quad LDS → RMIP → Bounded VNDF → Fast-MSX
- Material library: 5+ materials (stone, wood, concrete, ceramic, brick)
- Material editor: Live parameter editing with real-time preview
- Comparison modes: Toggle each technique on/off for ablation study
- Performance optimization: Per-technique profiling and optimization

Milestone 3 (Dec 1): Production Features

- Advanced features: Complete material authoring workflow, presets, export functionality
- Multiple test scenes: Showcase, performance stress test, quality comparison
- Comprehensive benchmarks: FPS vs. resolution, convergence analysis, memory comparison, quality metrics

Final (Dec 7): Polish & Documentation

- Code documentation, README, build instructions
- Final presentation materials + demo video (2-3 minutes)
- Open-source release with material library

References

Papers:

1. Thonat et al., "RMIP: Displacement ray-tracing via inversion and oblong bounding", SIGGRAPH Asia 2023 DOI: <https://doi.org/10.1145/3610548.3618182>
2. "Fast-MSX: Fast Multiple Scattering Approximation", SIGGRAPH 2023, <https://dl.acm.org/doi/10.1145/3610548.3618231>
3. Ostromoukhov et al., "Quad-Optimized Low-Discrepancy Sequences", SIGGRAPH 2024, <https://dl.acm.org/doi/10.1145/3641519.3657431>
4. Tokuyoshi & Eto (AMD), "Bounded VNDF Sampling for Smith-GGX", SIGGRAPH Asia 2023 / I3D 202, <https://dl.acm.org/doi/10.1145/3651291>

Reference Implementations:

1. Quad LDS: GitHub reference code (<https://github.com/liris-origami/Quad-Optimized-LDS>)
 - Used for generator matrix construction methodology
 - GPU adaptation and optimization is our original work
2. Bounded VNDF: AMD GPUOpen paper with pseudocode
 - Algorithm implementation is our original work
3. RMIP & Fast-MSX: No reference code available (algorithm from papers only)

Material Assets:

- CC0 texture libraries (ambientCG, Poly Haven)
- glTF sample models from Khronos Group